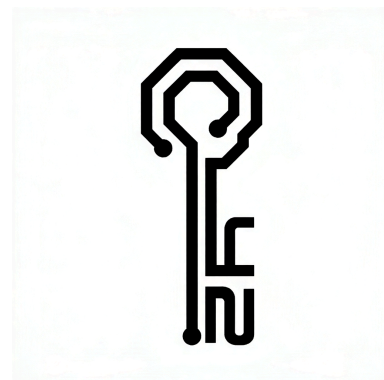


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



# ОТЧЕТ

**О выполнении лабораторной работы № 7  
«Совместное применение OpenMP и MPI»**

**Студент:** Доценко В. А.

**Группа:** Б23-505

**Преподаватель:** Куприяшин М.А.

Москва — 2025

# Оглавление

|                                           |   |
|-------------------------------------------|---|
| 1. Цель работы .....                      | 3 |
| 2. Рабочая среда .....                    | 3 |
| 3. Описание алгоритма .....               | 3 |
| 4. Используемые директивы OpenMP .....    | 5 |
| 5. Используемые функции MPI .....         | 5 |
| 6. Результаты экспериментов .....         | 6 |
| 6.1. Дополнительное задание .....         | 6 |
| 7. Заключение .....                       | 8 |
| 8. Приложение .....                       | 9 |
| 8.1. Разработанные программные коды ..... | 9 |

# 1. Цель работы

Приобрести навыки совместного применения технологий OpenMP и MPI для решения масштабных вычислительных задач.

# 2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (`_OPENMP = 202011`).

# 3. Описание алгоритма

1. Разбиение диапазона по процессам. Для каждого процесса вычисляются локальные границы `[local_a, local_b]`
2. Параллельная проверка чисел внутри интервала. Внутри каждого MPI-процесса запускается параллельный регион OpenMP, внутри которого числа проверяются на простоту.
3. Передача результатов на корневой процесс и сбор результатов в корневом процессе в общий массив простых чисел.
4. Вывод результатов.

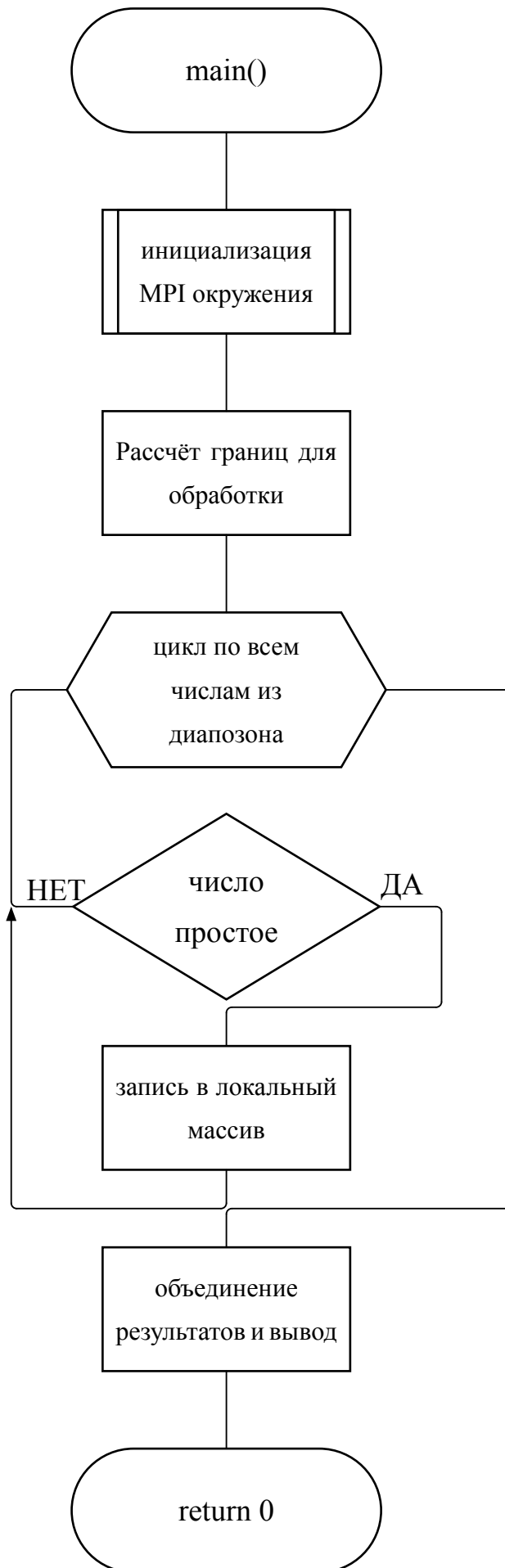


Рисунок 1 – Блок-схема алгоритма поиска простых чисел в диапазоне

## 4. Используемые директивы OpenMP

- **omp\_set\_num\_threads()**

Задаёт число потоков, которые будут применяться в следующих параллельных областях.

- **#pragma omp parallel for schedule(dynamic, 100)**

Создаёт параллельный регион с распределением итераций цикла между потоками.

## 5. Используемые функции MPI

- **MPI\_Init()**

Инициализация среды MPI.

- **MPI\_Comm\_rank()**

Определяет номер (ранг) процесса внутри коммуникатора.

- **MPI\_Comm\_size()**

Сообщает общее количество процессов, участвующих в коммуникаторе.

- **MPI\_Bcast()**

Рассылает данные от одного (корневого) процесса всем остальным, обеспечивая синхронизацию и единое содержимое для всех.

- **MPI\_Gather()**

Позволяет собрать одинаковое число элементов от каждого процесса в единый массив на корневом процессе.

- **MPI\_Gatherv()**

Аналогично MPI\_Gather, но принимает разное число элементов от каждого процесса; требует указания размеров блоков и их смещений в итоговом массиве.

- **MPI\_Wtime()**

Возвращает текущее время в секундах.

- **MPI\_Finalize()**

Завершает работу MPI: освобождает ресурсы и гарантирует корректное завершение всех операций.

## 6. Результаты экспериментов

Таблица 1 – входные параметры при измерениях

| Кол-во процессов | Кол-во потоков | Размер диапазона                        | Принцип разделения |
|------------------|----------------|-----------------------------------------|--------------------|
| 4                | 2              | 200 млн элементов<br>( $2 - 2 * 10^8$ ) | На равные блоки    |

Таблица 2 – Сравнение последовательной и параллельной реализации

| Время послед., сек | Время MPI+OpenMP, сек | Ускорение | Эффективность |
|--------------------|-----------------------|-----------|---------------|
| 135.374            | 35.614                | 3.80      | 0.475         |

Таблица 3 – Времена работы процессов при параллельной реализации

| Процесс 0, сек | Процесс 1, сек | Процесс 2, сек | Процесс 3, сек |
|----------------|----------------|----------------|----------------|
| 15.4485        | 25.2425        | 31.306         | 35.6142        |

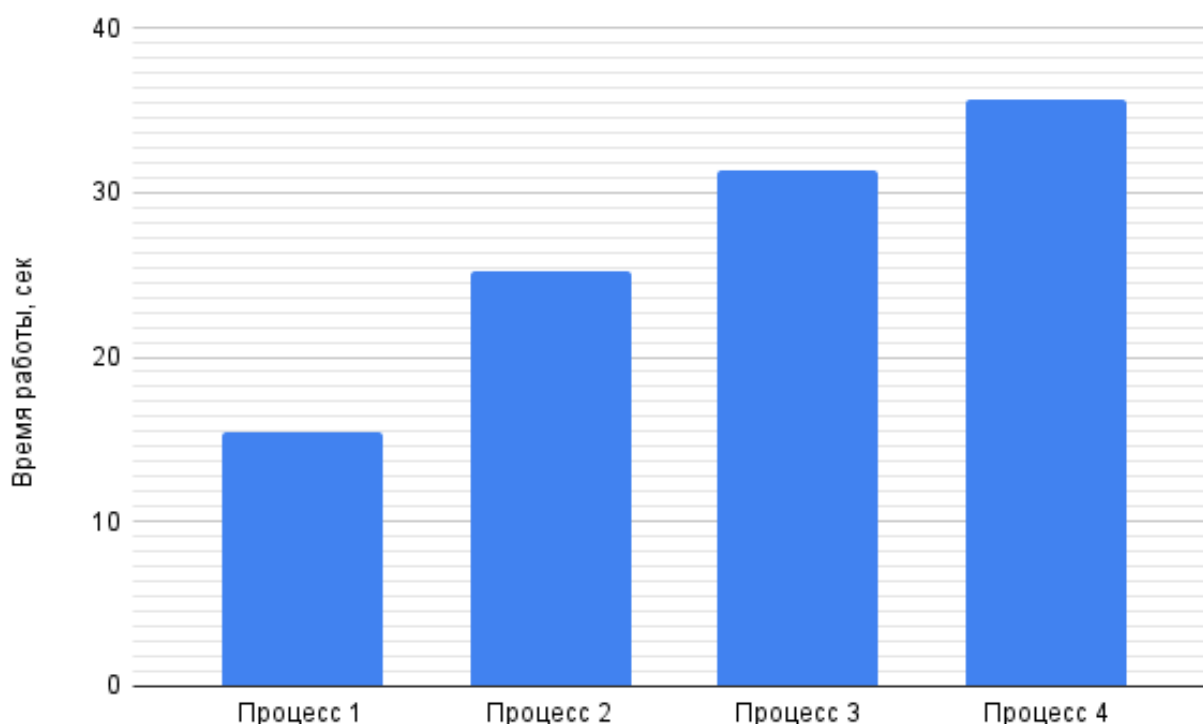


Рисунок 2 – Диаграмма распределения нагрузки между процессами

### 6.1. Дополнительное задание

Дополнительно был протестирован другой метод распределения нагрузки между процессами: нагрузка выравнивается за счёт циклического распределения *только нечётных чисел* между MPI-процессами, так что каждый процесс получает одинаковое количество вычислительно дорогих проверок простоты. Это

означает, что каждый процесс обрабатывает не подряд идущие числа, а числа, разделённые равномерно по всему диапазону. Например, если диапазон чисел разделён на 4 процесса, то первый процесс будет обрабатывать числа 1, 9 и т.д., второй процесс — 3, 11 и так далее.

Таблица 4 – входные параметры при измерениях

| Кол-во процессов | Кол-во потоков | Размер диапазона                        | Принцип разделения |
|------------------|----------------|-----------------------------------------|--------------------|
| 4                | 2              | 200 млн элементов<br>( $2 - 2 * 10^8$ ) | Циклический        |

Таблица 5 – Сравнение последовательной и параллельной реализации, для циклического распределения

| Время послед., сек | Время MPI+OpenMP, сек | Ускорение | Эффективность |
|--------------------|-----------------------|-----------|---------------|
| 135.374            | 27.215                | 4.97      | 0.62          |

Таблица 6 – Времена работы процессов при параллельной реализации, для циклического распределения

| Процесс 0, сек | Процесс 1, сек | Процесс 2, сек | Процесс 3, сек |
|----------------|----------------|----------------|----------------|
| 27,215         | 26,7488        | 27,1106        | 26,9265        |

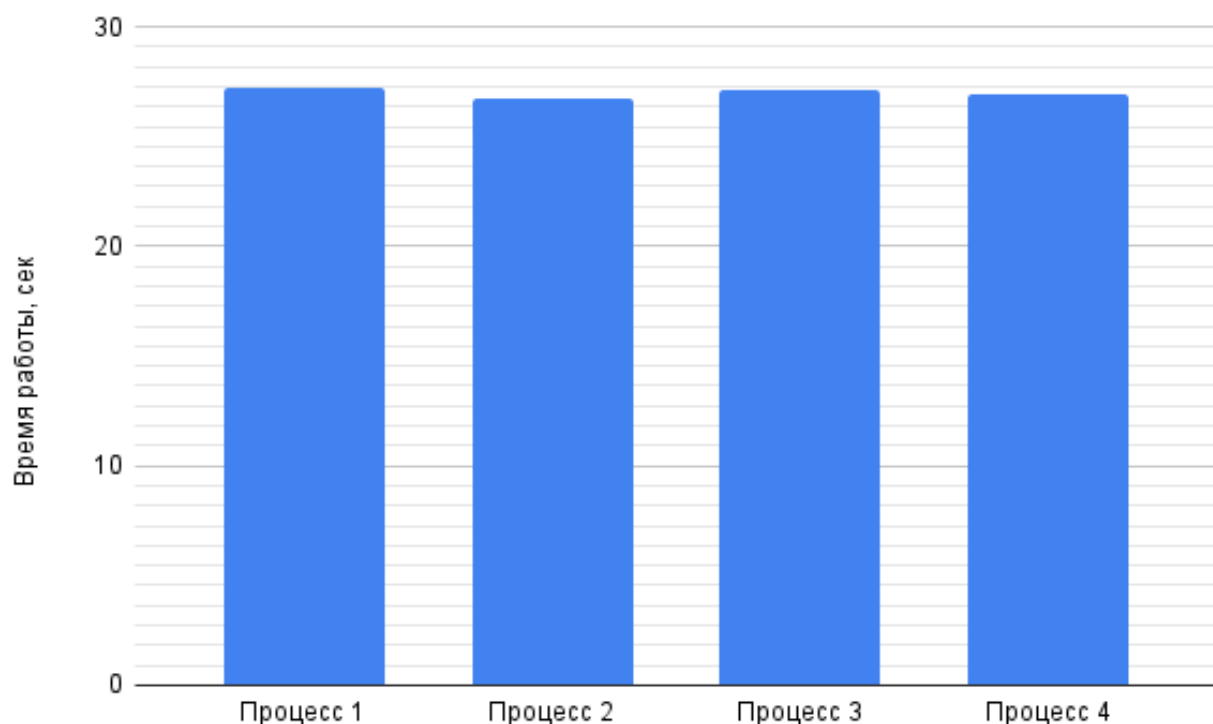


Рисунок 3 – Диаграмма распределения нагрузки между процессами, для циклического распределения

## 7. Заключение

В рамках лабораторной работы была разработана параллельная программа для поиска простых чисел с комбинированным использованием технологий MPI и OpenMP. Вычисления распределялись между 4 MPI-процессами, каждый из которых задействовал многопоточность OpenMP для обработки выделенного поддиапазона. Для оценки эффективности параллелизации была дополнительно реализована последовательная версия алгоритма. Были выполнены замеры производительности и анализ эффективности параллельной и последовательной реализаций. Для анализа использовался диапазон [2 – 200 млн.].

Результаты эксперимента показывают, что использование параллельного подхода позволяет снизить время выполнения задачи: последовательная реализация выполнялась  $\approx 135$  секунд, тогда как параллельная программа при использовании 4 процессов MPI и 2 потоков OpenMP на процесс завершила работу за  $\approx 36$  секунд. Полученное ускорение составило  $S \approx 3,8$ , а эффективность -  $E \approx 0,48$ .

Было измерено время работы по процессам. Результаты показали рост времени от первого процесса (15 сек) к четвертому (35 сек). Такой рост связан с тем, что проверка чисел на простоту в разных частях диапазона имеет разную вычислительную нагрузку, а простые числа неравномерно распределены по диапазону.

Полученные данные показывают, что совместное использование MPI и OpenMP даёт возможность более рационально распределять нагрузку между узлами и потоками.

**Дополнительное задание.** Протестирован метод циклического распределения нагрузки, при котором процессы получают равномерно распределённые диапазоны чисел с учётом их сложности обработки. Это привело к более равномерной нагрузке между процессами, что подтверждается схожими временными результатами работы процессов. Время работы составило  $\approx 27,215$ , ускорение достигло  $S \approx 4,97$ , а эффективность  $E \approx 0,62$ , что делает этот метод более эффективным по сравнению с блочным распределением.

## 8. Приложение

### 8.1. Разработанные программные коды

```
#include <cstdint>

bool is_prime(int64_t num) {
    if (num < 2)
        return false;
    if (num == 2)
        return true;
    for (int64_t del = 2; del * del <= num; del++) {
        if (num % del == 0)
            return false;
    }
    return true;
}
```

Файл 1 – Функция для проверки числа на простоту

```
#include <iostream>
#include <vector>
#include <chrono>
#include <fstream>

#include "prime.h"

int main(int argc, char** argv) {
    if (argc < 3) {
        std::cerr << "Usage: " << argv[0] << " Nstart Nend\n";
        return 1;
    }

    int64_t Nstart = std::stoll(argv[1]);
    int64_t Nend   = std::stoll(argv[2]);

    if (Nend < Nstart) {
        std::cerr << "Error: Nend must be >= Nstart\n";
        return 1;
    }

    const int64_t total_size = Nend - Nstart + 1;
    std::vector<int64_t> result_primes;
    result_primes.reserve(total_size);

    auto t0 = std::chrono::high_resolution_clock::now();

    for (int64_t num = Nstart; num <= Nend; num++) {
        if (is_prime(num)) {
            result_primes.push_back(num);
        }
    }

    auto t1 = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> dt = t1 - t0;
    double total_time = dt.count();
}
```

```

{
    std::ofstream fout("results/sequential_results.txt");
    if (!fout) {
        std::cerr << "Error: cannot open sequential_results.txt\n";
        return 1;
    }
    fout << "Время выполнения: " << total_time << " сек\n";
}

std::cout << "Sequential: " << total_time
           << " sec, primes: " << result_primes.size() << "\n";

return 0;
}

```

## Файл 2 – Программа для поиска простых чисел, последовательная реализация

```

#include "prime.h"
#include <mpi.h>
#include <omp.h>

#include <vector>
#include <fstream>
#include <iostream>
#include <algorithm>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int mpi_rank = 0, mpi_size = 1;
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);

    const int total_cores = 4;
    const int total_threads = 8;
    const int expected_mpi_processes = total_cores;
    int omp_threads = std::max(1, total_threads / std::max(1, total_cores));

    if (mpi_rank == 0 && mpi_size != expected_mpi_processes) {
        std::fprintf(stderr, "Warning: Expected %d MPI processes for this configuration, got %d\n",
                     expected_mpi_processes, mpi_size);
    }

    omp_set_num_threads(omp_threads);
    int omp_max_threads = omp_get_max_threads();

    int64_t Nstart = 1000000000000LL;
    int64_t Nend = 1000000001000LL;
    if (mpi_rank == 0) {
        if (argc >= 3) {
            Nstart = std::stoll(argv[1]);
            Nend = std::stoll(argv[2]);
        }
    }
}

```

```

MPI_Bcast(&Nstart, 1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);
MPI_Bcast(&Nend, 1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

int64_t total_numbers = Nend - Nstart + 1;
if (total_numbers <= 0) {
    if (mpi_rank == 0) std::fprintf(stderr, "Empty range: [%lld, %lld]\n", Nstart, Nend);
    MPI_Finalize();
    return 1;
}

int64_t block_size = total_numbers / mpi_size;
int64_t local_start = Nstart + mpi_rank * block_size;
int64_t local_end = local_start + block_size - 1;
if (mpi_rank == mpi_size - 1) local_end = Nend;

if (local_start > local_end) {
    local_start = 1;
    local_end = 0;
}

double proc_start = MPI_Wtime();

int64_t local_count_estimate = std::max(0LL, local_end - local_start + 1);
std::vector<std::vector<int64_t>> thread_bins(omp_max_threads);
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    auto &bin = thread_bins[tid];
    #pragma omp for schedule(dynamic, 100)
    for (int64_t num = local_start; num <= local_end; ++num) {
        if (is_prime(num)) {
            bin.push_back(num);
        }
    }
}

size_t found = 0;
for (auto &b : thread_bins) found += b.size();
std::vector<int64_t> local_primes;
local_primes.reserve(found);
for (auto &b : thread_bins) {
    local_primes.insert(local_primes.end(), b.begin(), b.end());
}

double proc_end = MPI_Wtime();
double proc_time = proc_end - proc_start;
int found_int = local_primes.size();

std::vector<double> procs_times;
std::vector<int> counts;
if (mpi_rank == 0) {
    procs_times.resize(mpi_size);
    counts.resize(mpi_size);
}

MPI_Gather(&proc_time, 1, MPI_DOUBLE,
          (mpi_rank == 0) ? procs_times.data() : nullptr, 1, MPI_DOUBLE,

```

```

    0, MPI_COMM_WORLD);

MPI_Gather(&found_int, 1, MPI_INT,
          (mpi_rank == 0) ? counts.data() : nullptr, 1, MPI_INT,
          0, MPI_COMM_WORLD);

std::vector<int> displs;
std::vector<int64_t> result_primes;
int total_found = 0;
if (mpi_rank == 0) {
    displs.resize(mpi_size);
    int pos = 0;
    for (int i = 0; i < mpi_size; ++i) {
        displs[i] = pos;
        pos += counts[i];
    }
    total_found = pos;
    result_primes.resize(total_found);
}

MPI_Gatherv(
    (found_int > 0) ? local_primes.data() : nullptr,
    found_int, MPI_LONG_LONG,
    (mpi_rank == 0) ? result_primes.data() : nullptr,
    (mpi_rank == 0) ? counts.data() : nullptr,
    (mpi_rank == 0) ? displs.data() : nullptr,
    MPI_LONG_LONG,
    0, MPI_COMM_WORLD
);

if (mpi_rank == 0) {
    double global_end = MPI_Wtime();
    double max_proc = *std::max_element(procs_times.begin(), procs_times.end());
    double global_time = max_proc;

    double sequential_time = 0.0;
    std::ifstream seq_file("results/sequential_results.txt");
    if (seq_file) {
        std::string line;
        while (std::getline(seq_file, line)) {
            double t;
            if (sscanf(line.c_str(), "Время выполнения: %lf сек", &t) == 1) {
                sequential_time = t;
                break;
            }
        }
    }

    double speedup = (sequential_time > 0.0) ? (sequential_time / global_time) : 0.0;
    double efficiency = 0.0;
    if (global_time > 0.0) {
        efficiency = speedup / (mpi_size * omp_threads);
    }

    std::ofstream fout("results/parallel_results.txt");
    if (fout) {
        fout << "ЭКСПЕРИМЕНТАЛЬНЫЕ ДАННЫЕ:\n";
    }
}

```

```

fout << "Количество MPI-процессов: " << mpi_size << "\n";
fout << "Потоков на процесс (OpenMP): " << omp_threads << "\n\n";

fout << "Время последовательного алгоритма: " << sequential_time << " сек\n";
fout << "Оценочное время параллельного алгоритма (max proc): " << global_time <<
" сек\n";

fout << "Ускорение (Speedup): " << speedup << "\n";
fout << "Эффективность (Efficiency): " << efficiency << "\n\n";

fout << "ВРЕМЯ РАБОТЫ ПРОЦЕССОВ:\n";
for (int i = 0; i < mpi_size; ++i) {
    fout << "Процесс " << i << ": " << procs_times[i]
        << " сек, простых чисел: " << counts[i] << "\n";
}
}

std::printf("Parallel (max proc): %.6f sec, speedup: %.2f, efficiency: %.1f%%, total_primes:
%d\n",
            global_time, speedup, efficiency, total_found);
}

MPI_Finalize();
return 0;
}

```

### Файл 3 – Программа для поиска простых чисел, параллельная реализация

```

#include "prime.h"
#include <mpi.h>
#include <omp.h>

#include <vector>
#include <fstream>
#include <iostream>
#include <algorithm>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int mpi_rank = 0, mpi_size = 1;
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);

    const int total_cores = 4;
    const int total_threads = 8;
    const int expected_mpi_processes = total_cores;
    int omp_threads = std::max(1, total_threads / std::max(1, total_cores));

    if (mpi_rank == 0 && mpi_size != expected_mpi_processes) {
        std::fprintf(stderr, "Warning: Expected %d MPI processes for this configuration, got %d\n",
            expected_mpi_processes, mpi_size);
    }

    omp_set_num_threads(omp_threads);
    int omp_max_threads = omp_get_max_threads();
}

```

```

int64_t Nstart = 100000000000LL;
int64_t Nend   = 1000000001000LL;
if (mpi_rank == 0) {
    if (argc >= 3) {
        Nstart = std::stoll(argv[1]);
        Nend   = std::stoll(argv[2]);
    }
}

MPI_Bcast(&Nstart, 1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);
MPI_Bcast(&Nend,   1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

int64_t total_numbers = Nend - Nstart + 1;
if (total_numbers <= 0) {
    if (mpi_rank == 0) std::fprintf(stderr, "Empty range: [%lld, %lld]\n", Nstart, Nend);
    MPI_Finalize();
    return 1;
}

double proc_start = MPI_Wtime();

int64_t first_odd = (Nstart % 2 == 1) ? Nstart : Nstart + 1;
if (first_odd > Nend) first_odd = Nend + 1;

int64_t local_first = first_odd + 2LL * mpi_rank;
int64_t local_count = 0;
if (local_first <= Nend)
    local_count = ((Nend - local_first) / 2) / mpi_size + 1;

std::vector<std::vector<int64_t>> thread_bins(omp_max_threads);
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    auto &bin = thread_bins[tid];
    #pragma omp for schedule(dynamic, 100)
    for (int64_t i = 0; i < local_count; ++i) {
        int64_t num = local_first + i * 2LL * mpi_size;
        if (num <= Nend && is_prime(num)) {
            bin.push_back(num);
        }
    }
}

size_t found = 0;
for (auto &b : thread_bins) found += b.size();
std::vector<int64_t> local_primes;
local_primes.reserve(found);
for (auto &b : thread_bins) {
    local_primes.insert(local_primes.end(), b.begin(), b.end());
}

double proc_end = MPI_Wtime();
double proc_time = proc_end - proc_start;
int found_int = local_primes.size();

std::vector<double> procs_times;

```

```

std::vector<int> counts;
if (mpi_rank == 0) {
    procs_times.resize(mpi_size);
    counts.resize(mpi_size);
}

MPI_Gather(&proc_time, 1, MPI_DOUBLE,
          (mpi_rank == 0) ? procs_times.data() : nullptr, 1, MPI_DOUBLE,
          0, MPI_COMM_WORLD);

MPI_Gather(&found_int, 1, MPI_INT,
          (mpi_rank == 0) ? counts.data() : nullptr, 1, MPI_INT,
          0, MPI_COMM_WORLD);

std::vector<int> displs;
std::vector<int64_t> result_primes;
int total_found = 0;
if (mpi_rank == 0) {
    displs.resize(mpi_size);
    int pos = 0;
    for (int i = 0; i < mpi_size; ++i) {
        displs[i] = pos;
        pos += counts[i];
    }
    total_found = pos;
    result_primes.resize(total_found);
}

MPI_Gatherv(
    (found_int > 0) ? local_primes.data() : nullptr,
    found_int, MPI_LONG_LONG,
    (mpi_rank == 0) ? result_primes.data() : nullptr,
    (mpi_rank == 0) ? counts.data() : nullptr,
    (mpi_rank == 0) ? displs.data() : nullptr,
    MPI_LONG_LONG,
    0, MPI_COMM_WORLD
);

if (mpi_rank == 0) {
    double global_end = MPI_Wtime();
    double max_proc = *std::max_element(procs_times.begin(), procs_times.end());
    double global_time = max_proc;

    double sequential_time = 0.0;
    std::ifstream seq_file("results/sequential_results.txt");
    if (seq_file) {
        std::string line;
        while (std::getline(seq_file, line)) {
            double t;
            if (sscanf(line.c_str(), "Время выполнения: %lf сек", &t) == 1) {
                sequential_time = t;
                break;
            }
        }
    }

    double speedup = (sequential_time > 0.0) ? (sequential_time / global_time) : 0.0;
}

```

```

double efficiency = 0.0;
if (global_time > 0.0) {
    efficiency = speedup / (mpi_size * omp_threads);
}

std::ofstream fout("results/parallel_results_cyc.txt");
if (fout) {
    fout << "ЭКСПЕРИМЕНТАЛЬНЫЕ ДАННЫЕ:\n";
    fout << "Количество MPI-процессов: " << mpi_size << "\n";
    fout << "Потоков на процесс (OpenMP): " << omp_threads << "\n\n";

    fout << "Время последовательного алгоритма: " << sequential_time << " сек\n";
    fout << "Оценочное время параллельного алгоритма (max proc): " << global_time <<
" сек\n";
    fout << "Ускорение (Speedup): " << speedup << "\n";
    fout << "Эффективность (Efficiency): " << efficiency << "\n\n";

    fout << "ВРЕМЯ РАБОТЫ ПРОЦЕССОВ:\n";
    for (int i = 0; i < mpi_size; ++i) {
        fout << "Процесс " << i << ": " << procs_times[i]
            << " сек, простых чисел: " << counts[i] << "\n";
    }
}

std::printf("Parallel (max proc): %.6f sec, speedup: %.2f, efficiency: %.1f%%, total_primes:
%d\n",
            global_time, speedup, efficiency, total_found);
}

MPI_Finalize();
return 0;
}

```

Файл 4 – Программа для поиска простых чисел, параллельная реализация, циклическое распределение